USER MANUAL

CrystalVista

A single-file, browser-based tool for building crystal structures from CIF/SHELX data, analysing host–guest porous frameworks, and mapping the void and channel space inside them.

This manual documents every panel and control in the app, the macro/automation system, and — in the appendices — the algorithms behind the void/channel and molecular-surface calculations in more technical depth than the in-app Guide covers.

Covers CrystalVista.html, revision current as of 2026-10-08

Generated from the app's own in-app Guide plus source-level documentation

Source file: CrystalVista_Manual.html (kept alongside the app; regenerate the PDF after editing)

Contents

1 Introduction

2 Getting started

3 View and camera controls

4 Loading structures, symmetry, and growth

4.1 Loading a CIF, .res, .ins, or .xyz file

4.2 The Grow panel: Mouse, Pack, Symmetry, Molecular neighbourhood

4.3 Host and guest layers

5 Selecting, editing, and deleting atoms

6 Selection & geometry, and selection groups

7 Appearance: elements, atom style, and bonds

8 Display settings and Walk mode

9 Void and channel analysis

10 Molecular surfaces

11 Coordination polyhedra

12 L.S. planes

13 Macros and batch processing

14 Visual effects, rendering, and export

15 Orca bridge, cube files and the electrostatic potential

16 Difference electron density from a .fcf file

APPENDICES

A Keyboard shortcut reference

B Macro command reference

C How void/channel space is calculated (technical)

1. Introduction

CrystalVista is a single self-contained HTML file — no installation, no server, no build step. It opens in any modern desktop browser (Chrome or Edge give the fullest feature set, since they support the File System Access API used for folder-based batch processing and direct-to-disk saving; other browsers fall back to ordinary downloads). It is built for crystallographers and materials researchers working with porous host structures — metal–organic frameworks, zeolites, inclusion compounds, clathrates — who need to build a structure out from its asymmetric unit by crystallographic symmetry, separate a host framework from an included guest, and quantify the empty space (voids and channels) inside it.

Three ideas run through the whole app and are worth knowing before anything else:

- **Host and guest are independent layers.** Loading a CIF/.res/.ins file gives you a host. A guest (an included solvent molecule, a gas, a templating cation) is created by selecting atoms already present and moving them into their own guest layer — there is no separate guest file. Deleting, hiding, or recolouring one layer never touches the other.
- **Growth, not a fixed molecule.** A loaded CIF shows only its asymmetric unit at first. Everything else — a whole molecule, a coordination sphere, a full unit cell, an extended framework — is built outward from that by explicit symmetry operations (the Grow panel), and can be undone or deleted again like any other edit.
- **Void/channel analysis works on whatever is currently host.** Guest atoms are always excluded from the void/channel search, so an included guest never masks the cavity it sits in. The calculation can run either on a symmetry-completed unit cell (the normal case) or on whatever host atoms happen to be grown and visible at the time (a narrower, non-periodic mode for exploring a partially built structure).

This manual follows the app's own panel layout (Structure, Style, Display, Void/Channel Analysis, Molecular Surface, Selection & Geometry, Macro, Visual Effects, Export) and adds three appendices: a full keyboard shortcut table, a full macro command reference, and a technical description of the void/channel algorithm for anyone who wants to know exactly what a reported volume means.

2. Getting started

Open `CrystalVista.html` in Chrome or Edge. There is nothing to install and no internet connection is required once the file is open.

1. **Load a structure.** Drag a `.cif`, `.res`, `.ins`, or `.xyz` file onto the 3D view. This becomes the **host** and, for a CIF or SHELX file, starts symmetry mode: only the asymmetric unit is shown at first. An `.xyz` file has no symmetry information to start from, so it loads as a plain, already-complete molecule instead.
2. **Grow the structure.** Open **Structure > Grow** and either click atoms with grow mode on (Mouse tab), or use Pack/Symmetry for an explicit range or operation, until the structure looks the way you want — a single molecule, a coordination sphere, a full unit cell, or an extended framework.
3. **Separate a guest, if there is one.** Select the atoms belonging to the guest (e.g. a solvent or templating molecule) and use **Move selection to > New guest group** (Structure > Edit atoms).
4. **Style it.** Use **Style > Atoms** and **Style > Bonds** for colours, radii, and bond thickness; **Display** for the unit cell, camera mode, and view direction.
5. **Analyse the void/channel space** from **Void / Channel Analysis** — see Section 9.
6. **Save your work. Export > Save model…** writes everything (both atom layers, symmetry data, void/channel results, display settings, camera position) to a single JSON project file that can be reopened exactly where it was left off.

Help buttons. A small **?** in the title bar of some menus and submenus (Structure Navigator, Add guest, Clipping Planes, Walk mode, Grow, Void / Channel Analysis, Molecular Surface, Orca bridge, Cube & ESP, Macro and Video) opens a short help panel for just that menu: what it is for, the steps, and the common snags. The panel looks like the Guide but floats beside the 3D view without dimming it, so it can stay open while you work. Drag its title bar to move it; press the **?** again, the × or Esc to close it; the link at its foot opens the full Guide. Clicking the **?** does not open or close the menu itself, and clicking in the panel does not cancel an interactive mode that is on.

Settings. The **Settings** button in the title bar, next to Elements, chooses which menus are shown, so the panel can stay simple. Seven groups can each be switched on or off: **Void / channel analysis**, **Molecular surface**, **Clipping planes**, **Host–guest tools** (the Host, Guest and Add guest submenus of Structure), **Walk mode**, **Macro and batch**, and **Quantum chemistry (Orca)** (the Orca bridge and Cube & ESP menus) and **Difference density (FCF)** (the Difference density menu, section 16, which shows only while a .fcf file is loaded with the structure and is off in the Viewer and Void / channel presets). The presets **Viewer**, **Void / channel** and **Everything** set them all at once. Every other menu is always shown.

Hiding a group only removes its menu. Nothing is switched off: keyboard shortcuts and macros still work, and the 3D view is unchanged. A group whose menu the loaded model needs is shown again for as long as it is needed, with a brief highlight, whatever the setting: void or channel results, molecular surfaces, guest components, a clipping plane that is switched on, a cube file or Orca results, a macro in

the memo, or Walk mode in use. So a saved model file always opens with the menus it needs, and nothing on the screen is left without a menu to manage it. Settings also holds the **default probe radius** for new molecular surfaces and a switch for the **?** help buttons.

A browser with no saved choice starts with the Void / channel preset. A browser that already holds CrystalVista preferences from before this feature existed starts with Everything, so no menu disappears. The choices are kept in the browser, like the element and display preferences, and are not part of saved model files; **Reset to defaults** returns to the Void / channel preset. Menus always start collapsed, and when several structures are loaded at once (a folder, a zip, or a CIF with several structures) the Structure Navigator opens by itself.

> **TIP**
>
> A project JSON file is also the only format that remembers which atoms are host and which are guest — a plain CIF never does. If you plan to batch-process a folder of structures with a macro that needs that distinction, save each one as a project file first (see Section 13).

3. View and camera controls

The **name of the loaded file** is shown in the header, in line with the left edge of the model panel (shortened with an ellipsis when too long; hover for the full name). It follows whatever is loaded — a CIF, a saved model, another structure of a multi-structure file — and clears when the structure is closed; the same name is listed as *File* under Structure > Metadata.

Action	Effect
Left drag	Free rotation about the camera's own axes. "Rotation sensitivity" (Visual Effects > Scene) can slow this down for lining up a precise view.
Shift + left drag	Roll — rotate in the screen plane, for straightening a figure without changing what you're looking at.
Shift + Ctrl + left drag	Pan — the whole model and its centre of rotation move across the panel together, following the cursor. The model still rotates and zooms about its own centre (e.g. its centroid), which now sits off the middle of the panel and stays put on screen as you rotate. Every component moves together (atoms, bonds, unit cell, voids, channels, molecular surfaces, coordination polyhedra, L.S. planes, labels) because the whole view is shifted. A Ctrl + click that doesn't drag still selects a whole fragment. **Display > Centering > Recenter**, loading a structure, or applying any Centering choice restores the centred view.
Alt + left drag	Lasso select: every atom (host and guest) whose on-screen position falls inside the drawn outline is added to the selection. This is a screen-space test, not a true depth test — an atom hidden behind another is still selected if its projection falls inside the outline.
Ctrl + Alt + left drag	Lasso select whole fragments — each atom the outline touches brings its entire bonded fragment into the selection.
Right drag, vertical	Zoom (up = in, down = out). The mouse wheel is disabled for zooming.
Display > Immobilise view	A tick box that switches off left-drag rotation and right-drag zoom, so a view you have set up cannot be nudged by accident. A plain click still selects atoms. Shift + left drag (roll), Shift + Ctrl + left drag (pan), the cursor keys, Page Up/Down and the `-` and `=` keys keep working.
Arrow keys	Rotate in fixed 5° steps — left/right about the vertical axis, up/down about the horizontal axis. Hold Shift with any of the four to snap a full 90° instead.
Page Up / Page Down	Roll in 5° steps (Shift = 90°).
`-` / `=` / `+`	Zoom out / in one fixed step, same as nudging "Set scale".
`Delete` / `Backspace`	Delete the selected atoms, or cut the selected bond(s) if no atoms are selected.
`Ctrl` + `C`	Generate a centroid from the selected atoms.

Esc	Clear the whole selection.
Space	Select everything if nothing is selected, or clear the selection if anything is.
Shift + Space	Invert the atom selection.
Ctrl + Space	Invert the bond selection.
Ctrl + X	Hide the selected atoms.
Ctrl + Z	Undo the last undoable action (the same history as the Undo buttons).
Ctrl + U	Unhide everything.
a	Toggle "Show ASU only" (CIF-based structures).
c	Toggle the unit cell.
k	Toggle the axis key.
h	Toggle hydrogen display.
l	Toggle atom labels.
p	Create a least-squares plane through the selected atoms (3 or more), then clear the selection — same as Style > L.S. Planes > New plane.

These shortcuts respond only while focus is not inside a text box, dropdown, or button. A complete alphabetised version of this table is repeated in Appendix A.

Camera modes

Three mutually exclusive modes (Visual Effects > Scene): **Orthographic** (default) is a true parallel projection — unit cell edges viewed straight down an axis overlay exactly, at the cost of a known artifact where zooming in close enough can clip through a solid atom (inherent to true orthographic projection, not a bug). **Pseudo-orthographic** and **Depth scaling** both use a genuine perspective camera parked far away at a narrow field of view, eliminating that clipping at the cost of very slight perspective convergence; Depth scaling exposes the angle as its own slider (up to 25°) for a deliberately stronger sense of 3D depth on a large structure. The chosen mode and angle, the background colour and the whole lighting setup (Visual Effects > Lighting) are remembered between structures and when the app is closed and reopened, and loading a saved project file does not change any of them.

4. Loading structures, symmetry, and growth

4.1 Loading a CIF, .res, .ins, or .xyz file

Dropping a `.cif` onto the 3D view loads it as the host and starts symmetry mode — it replaces any previously loaded host and clears the current guest along with it. The main view always means a new host. A guest is created from the loaded structure by selecting its atoms and using **Move selection to > New guest group**, or added from a file in **Structure > Add guest**: drop a CIF, or a SHELX `.res` or `.ins` file, that holds just one guest molecule on its drop zone. Every atom of the file is taken (the Q peaks of a .res are left out), its cell is used only to turn the fractional coordinates into Cartesian ones, the PART numbers of a SHELX file are kept, and the molecule is centred at the origin ready to be dragged into place. A CIF holding more than one structure (e.g. a CSD/Conquest search result with several `data_XXXX` blocks) loads its first entry and shows a **Navigator** with prev/next buttons and a dropdown for switching between the rest.

A CIF that contains its RES. SHELXL writes the instruction file it refined from into the CIF (the `_shelx_res_file` item; older versions used `_iucr_refine_instructions_details`). When a CIF holds one, the Navigator lists two entries for that structure, one after the other and titled **Structure Navigator**: **name — CIF** and **name — RES (in the CIF)**; hovering over either entry says that the other is in the same file. This applies to a CIF with a single structure and to each structure of a CIF with several (an entry without an embedded RES is listed alone), and equally to CIFs found in a dropped folder or zip file. The CIF entry is the CIF as before. The RES entry is read as a SHELX file, so it has the `PART` numbers (see below), which make the bonding of a disordered structure much easier to interpret, while the space group and the other file data (density, temperature and so on) are taken from the CIF part, since a RES has no space-group symbol of its own. **Save CIF** is available for the CIF entry only.

You can also drop a **folder**, a `.zip` file, or **several files at once**. CrystalVista searches everything dropped — subfolders too, and zip files inside zip files — for structure files (`.cif` , `.res` , `.ins` and `.xyz`) and lists each structure in the Navigator, which is then titled **Structure Navigator** (**CIF Navigator** when everything found is a CIF with no embedded RES). A file gives one entry, named after the file, with its folder path added where two files share a name (hover over an entry for its full path); a CIF holding several structures gives one entry per `data_` block. The first structure loads at once, and the line under the title shows which entry you are on and where the collection came from. Files with no usable structure (a CIF without atom coordinates, a text file) are left out, and the status line says how many. A zip that holds no structure files is read as a saved project, as before. The limit is 5000 files or 400 MB.

A SHELX `.res` or `.ins` file works identically from that point on — both formats are read into the same unit cell + symmetry operators + asymmetric unit representation internally. The one difference: **Save CIF** stays disabled afterwards, since there is no original CIF text to splice edits back into. A `PART n` instruction is read and remembered per atom: automatic bonding never bonds two atoms with different, both non-zero PART numbers (SHELX's convention for mutually exclusive disorder

components); a negative PART number additionally suppresses bonding, across symmetry, to a grown copy with any non-zero PART number. Shift + Ctrl + Alt +click selects every displayed atom sharing the clicked atom's PART number (SHELX-derived structures only).

Dropping a CIF reads the unit cell, the symmetry operators from the file's own symmetry loop, and the asymmetric unit, then displays just the asymmetric unit. If the file actually contains a whole completed molecule (some CIF exporters do this), the extra symmetry-equivalent copies are detected and removed automatically. A copy is removed only when it has the same element as an atom already kept and lies within 0.03 Å of a symmetry image of it, so the remaining set is the independent set as far as this proximity test can tell; closely spaced disorder sites (further apart than 0.03 Å) are kept as separate atoms, and the status line reports how many atoms were removed. An atom on a special position (mirror, inversion centre, rotation axis) is never duplicated when a symmetry operation maps it onto itself or an already-present copy.

CIF data names are read without regard to case (`_CELL_LENGTH_A` and `_cell_length_a` are the same), and both the traditional (`_atom_site_fract_x`) and the dotted (`_atom_site.fract_x` , including Cartesian-only files) spellings are understood. Numbers may carry an exponent (`1e-2`) and a standard uncertainty in brackets. An atom whose coordinates are missing or unreadable (`?` , `.`) is skipped and listed in the status line, and a cell that is incomplete or not a real cell is refused. Dropping any new structure, whatever its format, replaces the previous host, its guests, the undo history and all manual bond edits. In a SHELX file the free variables (`FVAR`) and PART occupancy codes are resolved into occupancies. **Save CIF** rewrites only the atom-site (and anisotropic) loops of the original file: columns the app does not edit keep their original values, and a coordinate, occupancy or displacement value you have not changed keeps its original text, standard uncertainty included; edited values are written without one, and a file that gives Cartesian coordinates is saved back in Cartesian coordinates.

An XMol `.xyz` file — a bare atom count, a title line, then one `element x y z` row per atom in Cartesian Ångströms — drops in the same way but loads as a plain molecule with no unit cell and no symmetry operators, since the format simply doesn't carry that information. The Grow panel, Fill unit cell, Forget symmetry, and cell-mode void/channel analysis are all unavailable for an XYZ-loaded structure, exactly as they are after Forget symmetry is used on a CIF/RES-derived one; Save CIF likewise stays disabled. Everything else — display style, bonding, selection, molecular surfaces, molecular-neighbourhood-mode void/channel analysis, export — works normally.

4.2 The Grow panel

Grow appears once a CIF (or .res/.ins) is loaded, split into four tabs.

Mouse

The large button turns grow mode on or off. Two independent choices control how a click grows the structure: **Bonds** (the new atom must be covalently bonded to the clicked one, searched up to 5 unit cells away) or **Sphere of inclusion** (every symmetry-equivalent atom within a given radius, regardless of bonding — useful for a coordination sphere or a non-covalent solvent shell). Separately: **Atoms** adds just the newly connected atom; **Fragment** (default) adds the whole covalently-connected molecule within the asymmetric unit; **ASU** adds the entire asymmetric unit under whatever symmetry

operation connects it. The Fragment/ASU distinction matters when the asymmetric unit holds more than one physically separate molecule (e.g. host and guest together) — ASU grows both since they share the transform; Fragment grows only the one actually clicked.

Pack

Grows by an explicit fractional-coordinate range (a, b, c each 0–1 by default, extendable to −5…5) instead of a click, with a choice of whether a fragment qualifies when a fragment atom or its centroid falls within the range. Every matching symmetry-related copy of the asymmetric unit is added as a whole covalently-connected fragment in one step.

Symmetry

Applies one explicitly chosen symmetry operation to whichever atoms are currently selected. The dropdown lists every space-group operator as the CIF itself expresses them; choosing one fills in the 3×4 matrix (rotation coefficients, always −1/0/1, plus translations). Each row also has its own −a/+a, −b/+b, or −c/+c pair for an immediate whole-cell shift along that axis. **Use M^{-1}** carries out the matrix in reverse; **Select new** selects each generated image in place of the source atoms (which is what makes repeatedly pressing a shift button walk outward one cell at a time); **Keep source format** copies each newly generated image's colour/radius directly from the atom it was generated from.

Molecular neighbourhood

A preview-first way of growing, for a framework or polymer that could extend in more than one direction. Enter a sphere-of-inclusion radius and turn the mode on: every not-yet-displayed symmetry-generated atom within that radius of something already shown appears as a small preview sphere (black by default, or coloured like the real atom with "Black nodes" unticked), joined by a dotted line to whatever it's close to. Clicking one adds the whole connected fragment it belongs to (Fragment-unit growth), and the preview refreshes around the newly larger structure. Nothing about the preview spheres is kept when the mode is turned off.

Settings for new host / guest atoms

Two dropdowns in the Grow panel decide the look a newly grown atom starts with, separately for the host and guest layers. **Default** leaves no override (plain ball-and-stick, CPK colour). **Current style** gives every new atom whatever Style > Atoms' Colour and Radii checkboxes currently hold. **Same as ASU** gives each new atom the look of its own asymmetric-unit parent atom — colour (CPK or custom), radius setting, and sphere-or-ellipsoid choice — so a symmetry copy always matches however its parent has been styled; a parent that has since been moved to the guest layer is still found, and an atom with no parent to copy from gets the Default look. The choice only applies at the moment of growth; atoms already present are never revisited.

Global growth helpers

Extend fragment(s) applies Mouse's Bonds+Fragment behaviour to every atom currently displayed in one step (only atoms present at the moment it is pressed are used as starting points, which is what stops a polymer or chain running away indefinitely). **Complete molecule(s)** fills out any partially-present fragment into a whole discrete molecule by symmetry, stopping cleanly at a genuinely polymeric/infinite fragment rather than trying to complete something that never ends; for a

coordination polymer it also tops up a bridging ligand truncated by ordinary growth without pulling in a new metal centre. **Generate whole molecules** (off by default) runs that same completion automatically after every growth action. **Delete all symmetry-generated atoms** removes everything growth has added, back to the bare asymmetric unit.

Forget symmetry permanently discards a structure's symmetry information: every atom present stays exactly where it is, but growth, ASU-only display, and all symmetry-linked cascades stop working, and every atom can then be edited completely independently. This cannot be undone and asks for confirmation.

4.3 Host and guest layers

Loading a new host CIF starts fresh, clearing any guest along with the old host. Within one loaded structure, host and guest stay independent: deleting or hiding atoms in one layer never affects the other. **Remove host** and **Remove all guests** each ask for confirmation, since for a CIF-based structure they permanently delete the relevant asymmetric-unit atoms and everything grown from them.

A structure can also be loaded as one combined file and split afterwards: select the atoms belonging on the other side, choose a destination (Host, an existing guest component, or "New guest group") from the dropdown, and press **Move selection to**. This is also how a disordered guest with more than one component is separated — move one component's atoms at a time into successive new guest groups. Guest components are listed on the **Guest** tab with an atom count, a show/hide checkbox, and a "Select" button for the whole component; an empty component (everything moved or deleted out of it) disappears from the list automatically, with the remaining ones renumbered.

Growth remembers a host/guest split: once part of the asymmetric unit has been moved to a guest, every future symmetry-generated copy of that same asymmetric-unit atom is added as a guest too, joining the same guest component rather than starting a new one each time.

5. Selecting, editing, and deleting atoms

Click an atom to select it (yellow wire highlight); click again to deselect. Modifier keys compose:

Click	Selects
Plain click	The one atom.
Shift + click	Every atom of that element, host and guest together.
Ctrl + click	The clicked atom's whole bonded fragment.
Alt + click	The clicked atom's whole component (host, or one specific guest group).
Shift + Ctrl + click	That element within the clicked atom's fragment only.
Ctrl + Alt + click	Every symmetry copy of that one asymmetric-unit position (CIF-based only).
Shift + Alt + click	Every atom generated by the same symmetry operator as the clicked atom, regardless of source (CIF-based only).
Shift + Ctrl + Alt + click	Every atom sharing the clicked atom's SHELX PART number (.res/.ins only).

Every one of these toggles based on whether the clicked atom is currently selected, so repeating the same click on an already-selected atom deselects the matching set. **Delete selection** removes the selected atoms from whichever layer they belong to.

Normalising hydrogen positions

In **Structure > Edit bonds**, **Normalise H atoms** corrects the hydrogen positions of an X-ray structure. X-ray H atoms sit about 0.1 Å too close to their parent atom, which matters for anything that measures space next to H (voids, channels, close contacts). Each H atom is moved along its existing X–H direction to a typical neutron-diffraction distance: C–H 1.083, N–H 1.009, O–H 0.983, B–H 1.19, S–H 1.34 Å (after Allen & Bruno, *Acta Cryst.* B66, 380, 2010). The five boxes above the button hold these values, can be edited, and are remembered between sessions.

The parent is the nearest non-H atom within 1.45 Å, symmetry images included. An H with two parents at similar distances (a bridging H), with no parent in range, or on a parent element with no distance set, is left unchanged and the status line reports how many. The command acts on the selected H atoms, or on **every** H atom when none is selected; **Undo** restores the old positions. For a CIF-based structure the change is made on the asymmetric-unit site, so every symmetry copy moves together and the symmetry stays exact. The macro command is `NORMALISE_H`.

Cutting and forcing bonds

Click directly on a bond to select it, then **Cut selected** to sever it — the one tool for the case nothing automatic can fix: two atoms sitting a normal bond length apart that shouldn't actually be connected (most often two overlapping disorder components). For a CIF-based structure, cutting a bond cuts

every symmetry-equivalent copy of it too. **Delete matching bonds** does the same in bulk by element pair and distance range. The **Interactive bond editor** (Structure > Edit bonds) additionally allows forcing a bond into existence between two atoms that aren't actually close enough to bond, by clicking them in sequence.

Show H bonds

When on, every displayed atom is scanned for probable hydrogen bonds, drawn as a short red dashed line. This is a geometric estimate (donor: a hydrogen bonded to exactly one N/O/S/Cl; acceptor: any other N/O/S/Cl within a plausible, roughly linear contact distance) — the same two-criterion first pass tools like Mercury use, not a quantum-mechanical calculation.

Mode buttons

A few buttons do not act once but switch an interactive mode on and off: Interactive bond editor, Click to add atom and Move atom (Create/Move atoms), Drag/reorient guest, Interactive grow mode (in Mouse, and again in Molecular neighbourhood), Move, Copy and Recolour (Void/Channel Analysis) Walk through structure (Display > Walk mode), and Record and Pause (Macro; one of the two is always on). They are shaded warm grey with a small hollow ring in front of the text, unlike the plain outlined buttons that act at once. While a mode is on the button turns red and the ring is filled; press it again, or another mode button, to leave the mode.

Undo, symmetry consistency, and special cases

A single shared undo history (Structure > Edit atoms or Edit bonds, either button) steps back through moving atoms, deleting atoms, and cutting bonds. Ctrl+Z does the same from the keyboard (the Windows convention). It does not cover loading files or element/display settings, which have their own separate resets.

For a CIF-based structure, an atom's element and its host/guest membership are properties of the asymmetric-unit *position*, not of one particular symmetry image of it — changing either on any one image applies automatically to every other currently-displayed image of that position. Deletion is the one exception handled two ways on purpose: deleting the untransformed (asymmetric-unit) copy of a position removes it entirely, including every symmetry image, and growth will never regenerate it; deleting a symmetry-grown copy on its own is a purely local edit — the asymmetric-unit atom stays, and growing again nearby can recreate that same image later.

Show ASU only (or the a key) is a display filter only — it hides every symmetry-grown copy, has no effect on what is calculated or grown, and clears the selection on anything it hides.

Select two or more atoms and **Generate centroid** (or Ctrl+C) adds a small white marker at their average position, useful as a geometric reference; it can be selected, moved between host and guest, and deleted like any real atom, and (for a CIF-based structure) becomes part of the asymmetric unit for growth purposes. **Reassign element to** changes the element of the selected atom(s) to anything in the periodic table (picking up its own colour/radius immediately); reassigning a centroid turns it into a real atom of that element.

6. Selection & geometry, and selection groups

The **Selection & geometry** section shows a live readout for the current selection. More than 4 atoms selected shows a count and an element breakdown; for 1–4 atoms it shows the last-selected atom's element and coordinates, then builds up cumulatively — 2 atoms adds the distance between them, 3 adds the second distance and the angle at the middle atom, 4 adds the third distance, second angle, and the torsion through all four — sighting from the second atom to the third, positive means the near bond needs a clockwise turn to eclipse the far bond (the standard IUPAC convention, matching Mercury, Olex2, PLATON, and SHELX). The order used is the order the atoms were actually selected in — click them one at a time in the sequence that gives the reading you want (e.g. along a bond chain for a meaningful torsion); deselecting one and reselecting it moves it to the end of the order. Where a CIF gives coordinate uncertainties, a distance/angle/torsion is shown with its own propagated esd (untransformed atoms only, no cell-parameter or cross-atom covariance).

Selection groups

Remembers a named set of atoms so it can be re-selected later without picking them by hand again. Select whatever's wanted (host and guest can be mixed in one group), optionally name it, and press **Save selection as group** — this also clears the selection just saved, since the new group's own **Select** button brings the same atoms straight back whenever wanted. Each group row shows an editable name, a live count of how many of its atoms are still actually present, and **Select**/**Deselect** buttons that add/remove just that group's atoms without disturbing an unrelated selection in progress, so several groups can be combined by selecting each in turn. An atom can belong to any number of groups at once, independent of host/guest membership; once every atom a group ever referred to has been deleted, the empty group disappears automatically. Selection groups are saved as part of the project file, the same as guest groups.

7. Appearance: elements, atom style, and bonds

Elements: radii and default colours

The **Elements** page is a periodic table showing every element's van der Waals radius and default colour together, plus D and T (deuterium/tritium) for neutron structures that label them as their own species. Radius is shared between host and guest and used both for the atom display and the molecular-volume calculation. The built-in default is **Bondi** (A. Bondi, *J. Phys. Chem.* 1964, 68, 441); anything Bondi doesn't cover gets a flat 2.0 Å. **Use Alvarez radii** (S. Alvarez, *Dalton Trans.* 2013, 42, 8617) offers a published alternative covering most naturally occurring elements including transition metals. **Save/Load element settings** reads and writes a CSV (atomic number, radius, colour); radii and colours are also remembered automatically per-browser between sessions.

Atom style

Every atom is CPK-coloured by default unless given a Custom colour from **Style > Atoms**. **Colour** is CPK/Custom/neither (tri-state); **Radius** is Radii×vdW (separate Non-H and H multipliers) or a single Fixed radius, or neither — only a ticked property is touched by Apply. Three preset icon buttons set common combinations: **Stick** (fixed radius 0), **Ball & stick** (0.2×/0.15× vdW), **Spacefill** (1.0×/1.0× vdW) — Apply is still required afterwards. **Copy settings** loads a single selected atom's current look back into the checkboxes, for copying it onto another selection. The Guest tab's own **colour by** (Element/Component) sets guest colouring as a whole, with a per-atom Custom override still taking priority over either.

Bond thickness

Set directly in Å (default 0.12), independently for host and guest. Each atom sphere is sized from its own element's van der Waals radius unless overridden per atom, and is widened to match bond thickness if that would otherwise make it narrower than the bond feeding into it — a radius override of 0 collapses an atom all the way down to the bond's own thickness for a "stick-only" look. Bonds are drawn split-coloured (each half in its own atom's colour) by default, or a single colour per layer if chosen.

Unit cell and axis key

Style > Unit cell opens with a **Show unit cell** checkbox that echoes Display's checkbox of the same name and the `c` key — change any one and the others follow. Below it are **Label axes** (the a/b/c corner labels), **RGB axes** (colour the a/b/c edges red/green/blue), **Cell colour** and **Line thickness** — all of these, plus whether the cell is shown, are remembered between structures and when the app is closed and reopened. Each corner label sits genuinely at its corner in 3D and is occluded normally by anything nearer the camera, not painted on top of everything. A divider then separates the cell's settings from the **axis key's**: **Show axis key** (echoing Display's checkbox and the `k` key), **Axis key zoom** (scales the whole key — arrow length and thickness, arrowheads and letters alike), **proportional lengths** (arrow lengths follow the real a/b/c edge lengths; off = all equal), **show**

labels a, b, c (a letter beside each arrow tip in the arrow's own red/green/blue on a transparent background, scaling with the key's zoom and positioned smoothly so it never jumps as the model rotates; on by default and remembered between sessions; included in exported images along with the key) and **Reset key position**. All of the key's settings — whether it is shown, where it was dragged, its zoom, proportional lengths and labels — are remembered between structures and when the app is closed and reopened (a saved project file still restores its own).

Atomic labels

Style > Atomic labels (moved here from Display, where it was called Labels) holds **Show atom labels** — echoing Display's checkbox and the `l` key — plus **Contrasting outline**, **Font**, **Colour**, **Size**, and the buttons that label/unlabel the selected atoms and unfix dragged labels.

8. Display settings and Walk mode

The **Display** section holds general settings (Show ASU only, Show atom labels, Show unit cell, Show axis key, Show hydrogen atoms, Show H bonds, Immobilise view, background colour, Centering with its Recenter button, Set scale) plus the nested **Direction** panel (view shortcuts along each cell axis/plane, a general [hkl] field, and View along vector / View perpendicular to plane — available once exactly 2 or exactly 3 atoms are selected respectively). The appearance of the unit cell, axis key and atom labels lives under Style (Section 7); the unit cell, axis key and atom-label checkboxes here are echoes of the ones there.

Clipping planes

A clipping plane (**Display > Clipping Planes**, up to six of them in a list) cuts away everything on one side of a plane so that the inside of a structure, void or channel can be seen. The tick box beside a plane in the list switches it on and the **cut** slider moves it from the near edge of everything loaded (0: nothing clipped) to the far edge (100: everything clipped), so the range scales itself to the structure; **Invert** clips the far side instead. **Apply to** selects what is cut: host, guest, voids, channels, molecular surface and unit cell (atom labels always follow the plane). A surface that is not cut (Molecular surface unticked) still hides whatever it encloses, so tick Molecular surface, or hide the surface, to see inside it. The plane is either **perpendicular to the view**, turning with the model, or **perpendicular to (hkl)**, fixed in the crystal, with the indices read as a plane normal or as a direction [hkl].

Solid cut faces (off by default, in addition to the usual look; a remembered preference with its own colour swatch, grey by default) closes the cut faces of clipped objects with a flat colour instead of leaving hollow shells with the bonds visible through the walls. The colour appears only where an object is actually cut, never across the whole plane. The tick boxes below it choose the kinds of object that get a solid face: Atoms and bonds, Voids, Channels and Molecular surface; a kind only gets one if the plane also clips it (the Apply to boxes), and atoms and bonds follow the Host and Guest boxes. For atoms and bonds the colour is laid down where the line of sight passes from the clipped-away region into a solid, so overlapping atoms and bonds fill as one; for voids, channels and the molecular surface the face is the plane's actual section through the surface, which does not depend on the viewing direction, stops at the unit cell face for a channel, and covers the whole cross-section of a void. The face of a void, channel or molecular surface is drawn on both sides, so it can be seen from either side of the plane (through a translucent surface, or from the kept side of a surface that has been cut open); the face of atoms and bonds is solid, so from the kept side you see the atoms themselves, not the inside of the cut. Labels and the unit cell are never filled. The faces are included in exported images and videos. With Invert on the cut faces lie on the far side of the objects and are seen by turning the model round. A very small solid disc can appear where the plane just grazes an atom, most noticeably a white hydrogen on a white background: it is that atom's cut face.

Several planes. The **Clipping Planes** submenu holds a list of up to six planes. Each row has a tick box that switches that plane on or off, its name and a small chip showing the colour of its solid faces; click a row to select it. **Add plane** makes a new plane that starts with the selected plane's settings (so a parallel slab is one click and one position change), and **Remove plane** deletes the selected one. The

controls below the list — **name** (type to rename), cut slider, position and step, invert, solid faces and colour, Apply to, orientation — always belong to the selected plane, and every plane keeps its own set. Where several planes are enabled an object is cut away if any plane removes it, so planes together can cut out a slab, a wedge or a corner; each has its own solid colour, and a solid face never shows where another plane has already removed the object. The list, names and settings are remembered between sessions (which planes are switched on is not).

Fine positioning. One step of the slider is 1% of the structure's width, a few tenths of an Ångström. The **position** box under it gives the plane's distance from the centre of the view along its normal (negative = nearer the viewer); type a value or use its arrows, which move the plane by the increment chosen in **step** (1, 0.1, 0.01 or 0.001 Å). With the slider focused (as it is after a click on it) the **cursor keys** move the plane by the same step (default 0.1 Å), and **Shift** with a cursor key moves it ten times as far. The slider and the box are always in step.

Walk mode

A separate, free-fly first-person view for exploring the inside of a void or channel — a different way of looking at the structure entirely, not a variation on the ordinary orbit view. Turning it on drops the camera into the scene exactly where the orbit view was last looking; turning it off returns instantly to that same orbit view.

Control	Effect
Mouse wheel	Move forward/backward along the current facing direction.
Left-drag	Turn the look direction, purely on the spot (no orbit point, no pitch limit).
Right-drag	Pan/strafe in the screen plane without turning.
Shift + left-drag	Roll (tilt the horizon).

Both drags use the app's own "grab and drag the scene" convention (drag right = the view moves as if the structure were pushed right), matching the ordinary orbit view rather than a first-person-shooter's "push the mouse to turn your head" — **Reverse drag directions** flips looking-around (not panning) for anyone who prefers it the other way. **Recenter view** snaps back to the exact view Walk mode was entered from. **Walk speed**, **Pan sensitivity**, and **Swivel/roll sensitivity** are independent. **Headlamp** is a light that travels with the camera, off the instant Walk mode is left.

Collision detection (off by default) stops the camera at whatever is currently drawn, governed by its own "Collide with" checklist — Host/Guest atoms and bonds, Voids/Channels, Molecular surface, and Coordination polyhedra, all on by default. For void/channel surfaces specifically: collision matches exactly what is rendered on screen, including a clip-plane cut at a cell face (never a missing wall, occasionally a slightly larger one right at the boundary) and a periodic channel's own single rendered period — its own repeats are not conjured up just for collision, so a channel that has not been manually copied (Fragments panel > Move/Copy) will have walkable open ends where the next period would be.

9. Void and channel analysis

This is the app's core analytical feature: finding the empty space — **voids** (finite pockets) and **channels** (space that repeats endlessly along one or more cell axes) — inside a loaded structure, using van der Waals radii and a probe radius the same way the molecular-volume calculation does, but reporting the space itself rather than the atoms' own volume. Guest atoms are always excluded, so an included guest never masks the cavity it occupies.

Two modes

Fill unit cell silently completes the whole unit cell from the CIF's own symmetry and searches that, regardless of how much is actually grown on screen — this is the normal, periodic mode. **Use grown atoms only** instead searches only whatever host atoms are currently displayed, with no unit-cell boundary and no periodicity, so growing more or less of the structure changes the result, and a probe that can escape the grown structure entirely is never reported as an enclosed void.

Running a calculation

Calculate populates the results table (one row per region, with its stability limit, swept volume, and the surface itself) and draws the corresponding surfaces in the 3D view; **Clear all** removes them. Loading a new structure or project always clears previous results automatically. **Additive (keep previous results)**, unticked by default, controls whether a fresh Calculate replaces everything (the default) or adds its newly found regions to whatever is already on screen, leaving every earlier region's colour, visibility, and fragment state untouched — useful for comparing two probe radii side by side.

Reading the results table

Limit reports, for each component, how sensitive its void/channel classification is to the probe radius actually used. For a **channel**, shown as "≤ R" — the largest probe radius it would still be recognized as this same connected channel at; above it, part of the channel gets pruned away into separate void(s) or removed. For a **void**, shown as "≥ R" — the probe radius below which it would connect to a periodic image of itself and become a channel; a dash means it stays a void down to the practical floor tested (0.05 Å). In "Use grown atoms only" mode this is always a dash, since that mode has no periodic images to connect to.

The reported **volume** always has some grid-discretization error, more pronounced for a small cavity than a large one. **Accurate volumes (extrapolated, slower)** corrects for this — see Appendix C for exactly how — at the cost of roughly six times the calculation time.

Assemble & Consolidate (on by default) automatically reassembles a void/channel's edge/corner fragments into one whole piece nearest the origin and bakes that arrangement in permanently, the same as the Fragments panel's own manual buttons.

KNOWN LIMITATIONS

A very small void (a handful of nodes) can show a noticeably different reported volume between otherwise symmetry-equivalent copies of itself — grid-discretization noise, not a real difference; Accurate volumes resolves it. A channel periodic along more than one non-Cartesian-pure axis at once (a genuinely oblique/triclinic cell) can show a real, visible seam where two Fragments-panel pieces join end-on, and may need the Fragments panel used by hand rather than relying on Assemble & Consolidate.

Colour, opacity, fragments, and copies

Colour is set at three levels of precedence: a global "Same colours for all voids/channels" link, then a per-instance colour (an instance being the original or any Move/Copy/Recolour-made copy), then a fixed palette default. Opacity works the same three-level way, entirely independently of colour. A void/channel sitting at a cell edge or corner is shown as several complementary **fragments**, listed in the **Fragments** panel; **Assemble** moves them back onto each other, **Consolidate** bakes that into one permanent piece. **Move/Copy/Recolour** offers three mutually exclusive modes for repositioning, duplicating by translation, or recolouring fragments/instances selected directly in the 3D view (see the in-app Guide for the full click/Shift/Alt/Ctrl selection scheme, which mirrors ordinary atom selection).

10. Molecular surfaces

A different, simpler tool from Void/Channel Analysis: a solvent-accessible "contact" surface computed directly over whatever atoms are currently **selected** (host and guest combined), with no concept of enclosure, volume, or topology at all — it is a shell over a chosen set of atoms, not an analysis of the space around them.

Select atoms, set a **probe radius** (0.01–3.0 Å, default 1.0), and press **Calculate**: a distance-transform grid is built and the isosurface at exactly that probe radius is extracted — the outer contact shell a sphere of that radius would trace rolling over the selected atoms' van der Waals surfaces — then relaxed with alternating-sign smoothing and reprojected back onto the true isosurface to remove voxel-grid stipple without leaving gaps. Calculate is **additive**: each press appends a new surface over whatever is selected at that moment, so several surfaces over different sub-selections can coexist, each independently listed (with its own visibility checkbox, atom count, probe radius, and colour) and independently deletable. The selection is cleared automatically once a surface is calculated. A single global **opacity** slider and a **Same colour for all** link apply to every surface at once, or each can carry its own colour.

All computed surfaces are cleared automatically when a new structure is loaded, but they do round-trip through a saved project file, and integrate with Walk mode collision (Section 8) as a thin shell using the exact triangle data drawn on screen.

> **NOTE**
>
> A probe radius smaller than the grid's own step size risks a contact band thinner than one grid cell; the grid step is scaled down automatically as the probe radius shrinks (down to a 0.02 Å floor), so a very small probe radius costs more to compute rather than producing a broken surface.

11. Coordination polyhedra

Draws a polyhedron over the ligand atoms coordinating each visible metal centre. For every currently-displayed metal atom, the app finds its full symmetry-derived ligand set using the same covalent-radius bonding test as ordinary bonds, takes the convex hull of those ligand positions, and draws only the hull faces whose three vertex atoms are all actually on screen right now — so a partially grown structure shows a partial polyhedron rather than requiring the whole coordination sphere first. Coplanar ligand arrangements (square-planar, the equatorial plane of a trigonal bipyramid, etc.) are re-triangulated as a clean fan rather than left as degenerate slivers, and haptic/ring ligands (e.g. a cyclopentadienyl bound as a whole π-system) are detected and excluded from the vertex set. A manually cut or forced bond (Structure > Edit bonds) changes the ligand set the same way it changes ordinary bonding.

Turned on from **Style > Coordination Polyhedra**: a single show/hide toggle (off by default), an opacity slider, an edge-thickness slider for the solid cylindrical edge spines, and a choice between colouring each polyhedron by its own central metal's current display colour, or one single shared colour. There is no manual central-atom picker — every visible metal (anything not classified as a non-metal) automatically gets a polyhedron whenever the toggle is on and it has at least three co-displayed ligands. Polyhedra are only ever computed for a loaded CIF with intact symmetry information; nothing is drawn for a non-CIF structure or after "Forget symmetry". Polyhedra integrate with Walk mode collision and with the clipping plane's own separate Host/Guest apply toggles (split by the central metal's own host/guest membership, not the ligands'), and settings round-trip through a saved project.

12. L.S. planes

Show planes, the first control in **Style > L.S. Planes**, is a single master on/off (on by default) that hides every plane at once without deleting any of them or disturbing each one's own individual show/hide checkbox in the list below — turn it back on and every plane reappears exactly as it was left.

Select three or more atoms (host and guest together) and **New plane** (enabled the moment the selection count reaches three; or just press the `p` key, which does exactly the same) fits and adds a **least-squares mean plane** through them — the plane through their centroid whose orientation minimizes the summed squared perpendicular distance to every one of the fitted atoms, exactly the same "best-fit plane" convention Mercury, PLATON, and SHELXL all report, not simply a plane fixed by any three of the atoms with the rest merely tested against it. Mathematically, the fitted normal is the eigenvector of the atoms' own 3×3 covariance matrix with the smallest eigenvalue, found with the same symmetric-matrix eigensolver already used elsewhere in the app for ADP ellipsoid principal axes; the two largest-eigenvalue eigenvectors span the plane itself. The rendered shape is a polygon through each fitted atom's own projection onto the plane, angularly sorted around the centroid — select the six atoms of a ring and the plane reads as a hexagon with its own vertices sitting right where the atoms are (exactly there if they're perfectly planar, very close otherwise), not a generic bounding rectangle. The selection is cleared automatically once a plane is added, ready to select a different group of atoms for the next one.

Planes accumulate in a list rather than replacing the previous one, mirroring Molecular Surface's own list exactly: each row shows its atom count and **rms deviation** — the root-mean-square perpendicular distance of the fitted atoms from their own best-fit plane, 0 meaning perfectly flat — with its own show/hide checkbox and colour swatch, or tick **Same colour for all** to colour every plane at once instead of individually; a single **Plane opacity** slider applies to all of them together. **Clear all** removes every plane in one step, and each row's own `×` button removes just that one.

> **NOTE**
>
> A plane is a one-time snapshot of the fit at the moment **New plane** is pressed — like a molecular surface, it does not track or refit itself if the atoms it was fitted from are later moved, grown further, hidden, or deleted. Planes round-trip through a saved project file, but (unlike element radii/colours and other look-and-feel settings) are not remembered between browser sessions on their own.

13. Macros and batch processing

A macro is a plain-text sequence of commands, one per line, recorded live from clicks and button presses, typed or pasted in by hand, or a mix of both. It is designed to work not just on the structure it was recorded against but on a whole series of similar ones — e.g. a variable-temperature data set for the same crystal — provided each structure's asymmetric unit is labelled the same way and sits in the same place (nothing here can verify that automatically).

Recording and playing

Record/Pause are mutually exclusive; while Record is down, every action with a defined macro command is appended to the memo the instant it happens, in the exact sequence performed — not every action has one, and an unrecorded one is simply skipped, silently. **Save/Load** read and write a plain-text `.MAC` file (also droppable directly onto the 3D view). **Play** runs every line in order against whichever structure is currently loaded, reporting how many ran versus how many failed by line number — an unrecognized command, or an atom reference that doesn't match anything in the currently loaded structure, is skipped rather than stopping the whole run.

Two buttons insert a command with no live equivalent action, regardless of Record: **Add current POV** (a `VIEW` command capturing the exact camera orientation) and **Add EXPORT_PNG/Add SAVE_JSON** (their own suffix textboxes alongside).

Batch-processing a folder

Apply macro to all files in folder turns Play into a batch operation: grant access to a folder, then press Play as normal. Every `.cif` / `.res` / `.ins` / `.xyz` / `.txt` / `.json` file in that folder is loaded in turn (alphabetical order) and the whole memo runs against each one before moving to the next. `.json` files are project files previously saved from this app (Save model, or `SAVE_JSON` itself) — the only format that actually distinguishes host from guest, since a plain CIF never does; sorting a structure's atoms into host/guest once by hand and saving that as a project is how to build a folder of files a host/guest-aware batch macro can process. A `.zip` project archive is deliberately not picked up by the folder scan — only plain `.json`.

Exporting from a macro

`EXPORT_PNG` [*suffix*] saves the current view as a PNG using the current Export > Image settings, named from the structure's own name plus the given suffix. `SAVE_JSON` [*suffix*] writes the current model out as a project file, the same complete payload Save model writes, respecting the **Compress as zip** checkbox the same way Save model does. With no suffix it **overwrites** the exact file the structure was loaded from (falling back to the structure's own CIF-derived name if it wasn't loaded from JSON); with a suffix it saves alongside as `name_suffix.json` (or `.zip`). Both commands write into a folder granted once per browser session (the batch folder itself, if batch mode is on, otherwise a one-time folder picker) with no further prompts.

> **NOTE ON .ZIP OUTPUT AND BATCH INPUT**
>
> A `SAVE_JSON` that runs with **Compress as zip** ticked writes a `.zip` — which will **not** be picked back up by a later batch run over the same folder, since that folder scan only recognizes plain `.json`. Leave the box unticked for a "save now, batch-process again later" workflow.

CALCULATE_VOID and CALCULATE_VOID_Q

These press **Calculate** under Void/Channel Analysis exactly as if clicked by hand, using whatever probe radius and mode are on screen at the moment they play — with one deliberate exception. **Accurate volumes** is itself saved inside a project `.json`, so batch-processing a folder of them would otherwise have each file's own load silently reset that checkbox to whatever that particular file happened to have saved, making accurate-vs-quick effectively random across a batch rather than something the macro actually controls. `CALCULATE_VOID` therefore always **forces Accurate volumes ON** before calculating (and ticks the checkbox to match), regardless of what is currently set or what a just-loaded `.json` brought with it; `CALCULATE_VOID_Q` always forces it **OFF**, for the rarer case where a genuinely quick batch calculation is wanted. While Record is down, clicking the real Calculate button records whichever of the two matches the checkbox's own state at that moment, so replaying a recorded macro reproduces the exact accurate/quick choice that was live when it was recorded.

Each time either command plays, every region found is appended as its own row to the **Void/channel batch results** table in the Macro panel — Structure, ID, Type, Critical radius, Volume, and the probe radius in effect for that row — building one combined comparison table across everything processed so far this session. It is never cleared automatically; **Copy as TSV**/**Save as CSV** export it, **Clear table** empties it. An ordinary manual Calculate click never adds a row here.

NORMALISE_H

`NORMALISE_H`, when played, presses **Normalise H atoms** (Structure > Edit bonds), using the distances in the five boxes at the moment it plays. In folder-batch mode a whole series is one macro — for example `NORMALISE_H` followed by `CALCULATE_VOID` for every structure in a folder.

See Appendix B for the complete command reference.

14. Visual effects, rendering, and export

Visual Effects is split into three tabs: **Scene** (background, camera mode, detail level, fog, rotation sensitivity), **Material** (how atoms respond to light — roughness, metalness, reflection strength), and **Lighting** (key/fill/rim lights, each independently switchable, plus "Light follows view" and light angle). Each tab has its own "restore defaults" and its own "retain these settings" box (ticked by default, remembering that tab's settings between sessions per-browser).

High detail spheres & tubes swaps in finer tessellation at some cost to rotation smoothness for large structures. **Export image (PNG)** always renders at full tessellation regardless of that toggle, at a chosen resolution multiple, then restores interactive settings afterward. **Transparent background** applies to both image export and **Copy image to clipboard**. Images are made one at a time: a second Copy or Export pressed while the first is still being prepared waits for it. The clipboard is asked for as soon as the button is pressed, and is given the image when it is ready, so a large image no longer makes the browser refuse the copy; if the browser still refuses (the page needs focus and clipboard permission), the message says so. If **Show axis key** is on, it's drawn into the exported image too, at its current on-screen position, size, and orientation (a part overlapping the sidebar or header is clipped out, matching the live view).

Save model…/Load model… save or restore the complete working state — every atom, the CIF symmetry data, colour/radius overrides, bond/display settings, void/channel results, and the camera position — to a single JSON (optionally zip-compressed) file, distinct from the automatic per-browser memory of element radii and display preferences. This is the portable, permanent copy of a specific piece of work. Manually added and cut bonds are saved with it. The original CIF text is not part of a project, so **Save CIF** is unavailable after a project has been loaded; keep the source CIF (or save the CIF before saving the project) if you need it.

Video: rotation and rocking

Export > Video records the current view as a video. A choice at the top selects the **kind**: *Rotation* (one full 360° turn about a view axis, in the number of **steps** given) or *Rocking* (a swing back and forth, made to be played on a loop). The settings shown, and the label of the export button, follow that choice; **fps**, **size** and **format** (uncompressed AVI, or the much smaller MP4, which needs a one-time encoder download) apply to both. An axis is a view axis (+Y turntable, +X tumble, +Z roll) and a plus sign always means the right-hand-rule rotation of the model.

The rocking video

Setting	Meaning
Rock about	The axis; its sign is the direction of the first swing.
Angle	Degrees from the starting view to the turning point (fractions allowed).

One side / Both sides	One side: starting view → +angle → starting view. Both sides: −angle ↔ +angle through the starting view (a total sweep of twice the angle).
Momentum	On (default): the swing slows to a stop at each limit and starts back slowly, fastest in the middle (a pendulum, a sinusoid). Off: constant speed, reversing at once at each limit.
Frames per cycle	Frames in one complete swing out and back, moved to the nearest multiple of 2 (one side) or 4 (both sides) so the turning points fall exactly on a frame; the text under the settings reports any adjustment, the length of a cycle in seconds (frames / fps) and the largest step per frame.
Cycles in the file	The same rendered frames repeated. One cycle is enough for a video played on a loop.

The recording starts at the current view. **Looping.** The cycle consists of the N samples at phase k/N, k = 0…N−1, of a periodic curve: $\theta = A(1 - \cos 2\pi k/N)/2$ for one side, $\theta = A \sin 2\pi k/N$ for both sides (with momentum), or the corresponding triangle wave (without). The last frame is therefore one ordinary step before the first and is never a duplicate of it, so there is no repeated frame and no hesitation at the join in the second or any later cycle, and the speed is continuous across it (with momentum the acceleration is too; the one-sided swing is also at rest at both turning points). The angle reached is exactly the angle requested.

Preview and Stop. Preview rocks the view on the screen at the tempo of the video (frames per cycle divided by fps) and follows any setting changed while it runs, until **Stop**, when the view glides back to where it started. Closing the Video panel or starting an export also stops it.

Autorotate

Display > Autorotate turns the model on the screen without recording. **Continuous spin** keeps its original axis, direction and speed (°/s). **Rocking** swings back and forth, with the same one side/both sides and momentum options as the video, controlled by the **angle** and the **time per cycle** (seconds); it uses the axis and direction chosen above it (the sign is the direction of the first swing). **Play** rocks about the view you start from, **Stop** glides back to it, and dragging the model while it rocks moves the centre of the swing.

15. Orca bridge, cube files and the electrostatic potential

Two menus, **Orca bridge** and **Cube & ESP**, use the free quantum-chemistry program **Orca** as a separate number cruncher. CrystalVista writes Orca input files into a folder, you run them, and CrystalVista reads the results back. It never starts Orca itself, because a browser cannot run programs: each job comes with a small run script that you start by double-clicking. Orca is not part of CrystalVista and is not redistributed with it; it has to be installed on the computer (the scripts were tested with Orca 6.1 on Windows). Everything uses the coordinates exactly as they are in the view, so cube files and potentials from a job fall on the atoms that produced them.

Setup

Choose folder… selects the working folder; the dialog opens in the folder the structure was dropped from (Chrome or Edge are needed to write to a folder); each job gets its own subfolder, `pairs_<name>` or `mol_<name>`. **Method** offers a few presets (hybrid and range-separated functionals with D3BJ dispersion, and the composite r2SCAN-3c and PBEh-3c methods) and the Orca keyword line below it can be edited freely. **Counterpoise correction** is on for the first group and off for the composite methods. **cores** above 1 needs Microsoft MPI (`mpiexec`) to be installed, otherwise Orca stops at once; leave it at 1 if unsure. **MB** is the memory per core and **charge** the charge of each molecule (a dimer is twice this; an odd number of electrons switches the multiplicity to 2). **Orca command** is how the script starts Orca: `orca` if it is on the PATH, otherwise the full path to `orca.exe` . The settings are remembered between sessions.

Pair energies

1. Grow the neighbours of one molecule with the Grow panel, so that its surroundings are in the view. For hydrogen atoms from X-ray data, use Normalise H atoms first.
2. Select the central molecule (any atom of it, or Ctrl-click for the whole molecule) and press **Find dimers**. Molecules of the view that have an atom within the **contact cutoff** of the central one are its neighbours. Pairs that are the same by symmetry are listed once; the table gives how often each occurs (×), the centroid distance, the shortest contact and the neighbour's formula. Ticks choose what is calculated; a neighbour with a different formula starts unticked because a partial molecule at the edge of what was grown would give a wrong energy. Clicking a row selects that neighbour in the view.
3. **Write inputs** creates `dimer_01.inp` , `dimer_02.inp` , …, `run_pairs.bat` and `pairs_manifest.json` . Each input holds three Orca jobs: the dimer, then each molecule alone, so that E_{int} = E(dimer) − E(A) − E(B). With counterpoise on, each molecule is computed with its partner as ghost atoms.
4. Double-click `run_pairs.bat` . It skips jobs whose output already ends normally, so it can be restarted, and it deletes Orca's large scratch files afterwards.
5. Press **Read results**. The table shows E_{int} in kJ/mol (jobs that did not finish or did not converge are marked), and below it ½Σn·E: half the sum of the pair energies of the central molecule with its

neighbours, each counted as often as it occurs. It is a rough lattice energy per molecule; it contains no many-body terms and depends on the method and on how many neighbours were grown, so treat it as a comparison between structures of the same kind (for example along a pressure series) rather than an absolute value.

Energy framework draws a tube between the centres of every pair of molecules in the view that matches a calculated dimer, with a radius proportional to the pair energy (blue attractive, red repulsive); **tube radius at 50 kJ/mol** sets the scale and **smallest |E| shown** hides weak pairs. **Copy table** puts the table on the clipboard as tab-separated text.

Molecule job

Uses the selected atoms as one molecule and writes `mol_<name>.inp`, a single-point calculation, with any of: an **electron density cube**, **HOMO and LUMO cubes** (closed-shell molecules only), and the **electrostatic potential at every vertex of the molecular surfaces now in the view**. For the potential, build the molecular surface first (Molecular Surface). Building a surface clears the selection, so with the ESP box ticked and nothing selected the job uses the atoms the surfaces were built from; select atoms yourself to use a different set. `run_mol.bat` then runs Orca followed by its `orca_vpot` program on the surface points. **cube grid points** sets the resolution of the cubes (60 per side is a good start; the files are several MB each). **Read results** loads the cubes into Cube & ESP and colours the surfaces by the potential; the surfaces in the view must have the same atoms and probe radius as when the job was written, and a mismatch is reported. The potential belongs to the shape of the surface, not to the surface as an object: if you delete a surface and build it again with the same atoms and probe radius, it is coloured again at once, and Read results can be pressed again on the rebuilt surface.

Framework potential

The electrostatic potential of a host framework, mapped onto the wall of a channel or onto a guest molecule in it, shows where a guest is attracted or repelled. It is the kind of figure used to explain host–guest binding (for example Bezuidenhout *et al.*, *Angew. Chem. Int. Ed.* 2015, 54, 2079). This subsection of the Orca bridge builds it from **point charges**, which is quick and needs no Orca run; a calculation on a cluster of atoms with Orca is the planned, more accurate alternative.

1. **Prepare the structure.** It must be a CIF or SHELX structure that still has its symmetry. Only the **host layer** supplies charges, so move any guest into a guest group first (Structure > Edit atoms > Move selection to). Two atoms of the host closer than 0.9 Å are taken to be components of a disorder and stop the calculation: remove all but one component.
2. **Calculate charges.** Every host atom of the unit cell, all symmetry images, receives a charge from the **EQeq** charge-equilibration method (Wilmer, Kim and Snurr, *J. Phys. Chem. Lett.* 2012, 3, 2506), applied to the periodic structure with its published parameters (dielectric screening 1.2, hydrogen electron affinity −2.0 eV, non-neutral charge centres for metals, ionisation data as tabulated with the method). The cell is neutral and symmetry-equivalent atoms have equal charges. The table lists the charge of each atom of the asymmetric unit.
3. **Choose what to colour.** A **molecular surface** over the guest atoms gives the potential on the guest; over the host atoms around a channel (grow them with the Grow panel first, and use a small

probe radius such as 0.5 Å) it gives the potential on the channel wall, which is the priority case. The surfaces found by **Void / Channel Analysis** can be coloured as well, but they lie a probe radius away from the walls, so the potential on them is weaker. The two tick boxes choose between the kinds (Molecular surfaces, Void / channel surfaces) and take effect at once on a map that is on show.

4. **Map potential.** The potential of the host charges at every vertex is evaluated by Ewald summation of the periodic charges (so it does not depend on how much of the structure is grown), in kcal/mol, and painted on the surfaces. A colour bar appears in a corner of the 3D view; its span is set by **colour range** in Cube & ESP, and **Red = positive** there reverses the colours. The table gives the highest and lowest value on each surface, which is what is compared between frameworks.

Orca cluster: the potential of a real calculation

For a better potential, the host around a channel can be calculated with Orca as a finite **cluster** cut out of the crystal, which is what the paper cited above did (with a semi-empirical method). Orca then gives the electrostatic potential of the real electron density at every surface point, instead of the potential of point charges. The calculation runs on your own computer, with the cores set in Setup, in the same way as the other Orca jobs.

1. **Build the surfaces first** (a molecular surface over the guest, over the channel wall, or Void / Channel Analysis), as for the point-charge map. Their vertices are the points at which Orca will evaluate the potential.
2. **Build cluster.** Choose the **axis along the channel**, the number of **cells along it** (the paper used 3) and the **radius** of the cylinder around the channel axis, which passes through the centre of the guest atoms (or of the selected atoms if there is no guest). The host atoms of the cylinder are added to the view, as more of the host, and selected. They are completed so that the cluster stays neutral and closed-shell: only single carbon–carbon bonds (1.46 Å or longer) are cut, each capped with a hydrogen atom, and the dative bond from a metal to a nitrogen atom is cut without a cap; every other bond is kept, so a paddlewheel with all its carboxylate groups, or a whole aromatic ligand, is never split. You can add to or remove atoms from the selection before the next step, provided no other bond is cut. The line under the buttons gives the number of atoms and caps, an estimate of the number of basis functions and of the time, and the number of electrons (an odd number means a closed-shell calculation is impossible: change the charge).
3. **Write cluster job.** Writes the Orca input for the selected host atoms and their caps (the guest is left out), the surface points and `run_fw.bat` into a `fw_<name>` folder. The **keywords** default to `PBE def2-SVP def2/J RI`, a GGA functional with the resolution of the identity, which suits a large cluster; the calculation is **closed-shell** (multiplicity 1), so each copper(II) paddlewheel is treated as a closed-shell singlet, which is an approximation. At most **most points** surface vertices are sent to Orca; the others take the value of the nearest one, and the table then says that the extremes are approximate.
4. Double-click `run_fw.bat`. It runs Orca, checks that Orca finished normally (if not it stops and says so, instead of going on), and then runs `orca_vpot` on the points. For orientation: on the computer this was developed on (6 cores, the keywords above) a closed-shell cluster of 74 atoms (838 basis functions, one copper paddlewheel) converged in 17 SCF cycles and took 53 seconds, and a cluster of 261 atoms (3173 basis functions, twelve copper atoms) converged in 24 cycles and took 14

minutes; evaluating the potential at about 900 points took 6 seconds. The cost grows roughly as the square of the size, so a cluster of about 700 atoms is an estimate of two to three hours.

5. **Read cluster results.** If the Orca output does not end normally (for example because it ran out of memory or the SCF did not converge) nothing is coloured and the status line gives the reason, because a potential taken from an unfinished wavefunction is meaningless. The surfaces must be the ones that were in the view when the job was written (the same atoms, probe radius or Void / Channel settings); those that still match are coloured and listed with their highest and lowest values. Press **Choose folder…** and pick the folder that contains the `fw_` folder, or the `fw_` folder itself; if a file is missing the message names it and the folder that was searched.
6. **Add surfaces** (surfaces built after the job was run, for example over the guest). Orca evaluated the potential only at the points of the surfaces that were in the view when the job was written, so a surface built later has no values. Build it, press **Add surfaces**, and CrystalVista writes the points of every surface that has not been evaluated yet, together with `run_vpot.bat`, into the same `fw_` folder. Double-click `run_vpot.bat`: it evaluates the potential from the wavefunction (`.gbw`) that the finished job left in the folder, which takes minutes instead of hours; the SCF is not repeated. `orca_vpot` also needs the electron density (the `.densities` file), which jobs written by earlier versions deleted at the end. If it is missing, `run_vpot.bat` first rebuilds it from the converged orbitals with a copy of the input (`<name>_dens.inp`, keywords `MORead NoIter LooseSCF`: the density is written from the stored orbitals at the start and the SCF then needs only a cycle or two; the setup is repeated, so allow a quarter of an hour or more for a cluster of 700 atoms), and then evaluates the points. New jobs keep the density file. Then press **Read cluster results**: the surfaces of the original job and those added are coloured together. It can be repeated as often as needed. The job's `.gbw` file must still be in the folder (it is about 600 MB for a cluster of 700 atoms), so do not delete it.

CHOOSING THE CLUSTER

The cluster is a cut-out, so atoms near its ends and edges are missing their neighbours: the potential is most reliable near the middle of the cluster and close to the guest. Make the radius large enough to include the whole first ring of walls around the channel (for the frameworks above, about 8 Å for both sides of a channel), and the length long enough that the guest sits well inside it. Cost grows quickly with size, so build the cluster first and read the estimate before writing the job. Orca also needs memory in each core in proportion to the square of the number of basis functions (about 6 GB per core for 13,700 functions, about 1,250 atoms); the estimate gives the amount, **Write cluster job** raises MB in Setup to it if it is too small, and cores times that must fit in the computer's memory (use fewer cores or a smaller cluster if it does not). Setting up a cluster of that size took almost an hour before the SCF began.

WHAT THE NUMBERS MEAN

The potential is that of the periodic framework alone, with the usual convention that its average is zero, so absolute levels differ from those of a finite cluster. Point charges describe the potential well a little way from the atoms but not at close range, and EQeq charges are screened, so the values are smaller than those from a quantum-chemical calculation. Tested on the three copper(II) glutarate frameworks of the paper cited above, with CO_2 in the channels, the model reproduces the pattern the authors describe: the lowest potential on the CO_2 surface lies next to a carboxylate oxygen atom and the highest next to a framework hydrogen atom; and the potential on the CO_2 is much weaker in the framework with the shortest linker than in the other two. The magnitudes are smaller than those the authors obtained with a semi-empirical calculation on a cluster: at the lowest point of a molecular surface of the CO_2 (probe radius 0.5 Å; the 0.01 e/Å^3 density contour the authors used lies at about the same place, 1.6 to 1.7 Å from the nuclei of an isolated CO_2) the model gives −3, −19 and −18 kcal/mol for the three frameworks, against their −34, −58 and −62. Use the maps to compare sites and structures, not as absolute values.

Cube & ESP

Load cube file… (or drop a `.cube` file on the 3D view) shows a Gaussian cube file as an isosurface. Each file gets a card with its grid and value range, an **isovalue** (a sensible one is chosen; it is shown to three decimals and its spin buttons move by 0.001, or by a finer step for a cube whose values are all very small, such as a potential), colour swatches for the positive lobe and, for signed data such as orbitals, the negative lobe (each with its own tick box, to show or hide that lobe), and an **opacity**, which starts at 1 (fully solid); a translucent surface is drawn back faces first, then front faces, so it does not show stripes. Untick a card to hide the whole cube and use × to remove it, or **Clear cubes** to remove them all; loading a new structure removes them too. The surfaces are lightly smoothed to hide the steps of a coarse grid. A cube whose atoms do not coincide with the atoms in view is drawn in its own frame and says so.

ESP colouring paints the molecular surfaces by the electrostatic potential, either from the Orca results brought in by the bridge or from a loaded cube (**Colour surface**). Negative potential is red, zero white and positive blue, over the **colour range** in atomic units; the default covers 95% of the values. **Clear colours** restores the ordinary surface colours. The **surface opacity** slider in this submenu is the same setting as the one in Molecular Surface, so either one changes how see-through the coloured surface is.

REMEMBER

These are gas-phase calculations on the molecules as they sit in the crystal geometry: the pair energies leave out the rest of the lattice, the potential is that of the isolated molecule, and the results are only as good as the geometry (especially hydrogen positions) and the method chosen.

16. Difference electron density from a .fcf file

A SHELXL refinement can write the reflection data it used into a `.fcf` file. CrystalVista reads that file and calculates the **difference electron density** map, Fo − Fc, which shows where the electron density of the crystal is not explained by the refined model: the electron density of a guest that was left out of the model appears as a positive region, a wrongly placed atom as a negative one. The map is drawn as surfaces in the 3D view, in the same frame as the atoms.

Preparing the files

1. Refine the structure **without the guest** (delete its atoms from the model, then refine again), so that its density is left in the difference map.
2. Have SHELXL write the reflections with `LIST 6` (put that instruction in the .ins file; `ACTA` then writes the .fcf). A `LIST 6` .fcf has, for every unique reflection, Fo^2, its sigma, |Fc| and the phase of Fc, which is what is needed. The default `LIST 4` file has no phases and cannot be used yet: CrystalVista then says so.
3. Take the `.cif` and the `.fcf` of the **same** refinement. If the file was refined in a different cell setting from the CIF, CrystalVista refuses it (the cell is compared).

Loading

Drop the CIF and the .fcf **together** on the 3D view (several files at once, or a folder or zip file that holds both). A .fcf is paired with the structure file of the same name in the same folder; if only one structure file and one .fcf were dropped, they are paired whatever their names. When the CIF contains its RES, both entries of the Structure Navigator get the map. A .fcf dropped alone is attached to the structure that is already loaded. Saved model files do not keep the reflections: drop the .fcf again after loading a model.

Showing the menu

The **Difference density** menu is an optional menu. It appears only when **Difference density (FCF)** is switched on in **Settings** *and* a reflection file has been loaded with the structure; it disappears with the structure. If a .fcf is loaded while the setting is off, the status line says so and Settings notes it under the entry. Nothing is drawn while the menu is off.

The menu

- The first line names the file, the number of reflections and the resolution. Under the buttons are the highest and lowest values and the r.m.s. of the map (e/Å³); when the CIF holds SHELXL's own `_refine_diff_density_max/min/rms` they are shown beneath for comparison.
- **map**: the **difference** map (Fo − Fc, what the model lacks) or the **2Fo − Fc** map (the electron density itself, with the phases of the model, drawn in blue by default at 1 × its r.m.s., positive only). Each map keeps its own levels, colours and hidden parts, and the menu switches between them. The

2Fo – Fc map has no F(000) term, so it is contoured relative to its own mean, like the difference map; the highest and lowest values and the r.m.s. shown are those of the map on show. Counting electrons and the peak list always use the difference map, whichever map is on show, and the click of a count should be made on a surface of the difference map.

- **levels and surfaces**: up to three surfaces, at three levels (by default blue, green and red from the lowest level to the highest, with the negative density at the lowest level that is switched on, in a fourth colour). Each surface, the negative one included, has its own **style**: a wire **mesh** or a transparent **solid** with its own **opacity**. With suitable opacities every surface can be seen through the others (for example solids of 0.25 and 0.4 around a mesh), and the opacity can be changed while looking at the picture. The nested solids are drawn from the lowest level outwards in. The **spacing** sets how far apart the points of the mesh are (0.05 to 1 Å; a larger value gives a coarser, more open mesh, and 0.25 is the default). The **line width** slider sets the thickness of the mesh lines (0.5 to 6 screen pixels; it follows the pixel ratio, so a copied or exported image keeps the look). Each of the three levels has a checkbox to switch it on or off, a value, and a colour swatch (right-click a swatch for the favourite colours); the negative density is drawn at the lowest level that is switched on. A single transparent solid surface is simply one level switched on, with its style set to solid, the other two switched off.
- **levels in**: contour levels as multiples of the r.m.s. of the map (the usual 3σ style) or in e/Å³. Changing the unit converts the levels.
- **negative**: also shows the negative density.
- **region**: **box around the structure** draws the surfaces in a box around every atom in view plus the **margin**; **box around the selected atoms** does the same for the selected atoms only (select the atoms around a void to frame it; with nothing selected it is the whole structure); **near the visible atoms** and **near the selected atoms** draw them only within a **radius** of those atoms, which leaves out the density of the framework walls far from them. The map is periodic, so it is drawn for whatever is in view: use **Grow** to show the surroundings, then **Redraw**.
- **Erase** tidies the display. Press the red **Erase mode** button (like Grow > Mouse), choose a tool and click a surface of the map in the 3D view (a drag still turns the view, and atoms cannot be selected while the mode is on; press the button again to go back). The mode also turns off when you close the Erase submenu or the Difference density menu, or start another mouse mode such as interactive grow. **hide the blob** hides the whole connected blob of density under the pointer, **keep only the blob** hides everything else, and **hide a ball** hides everything within the chosen **radius** of the clicked point (see **copies** below). What is hidden is kept as rules about the map, not as pieces of the drawn surface, so the rules still apply when the level, the spacing or the region is changed: a blob is hidden as the density peaks it consists of (the map is divided once into basins, one for each peak), so if the level is lowered and a tail of density joins a hidden blob, the tail appears but ends where the hidden blob begins. The map repeats from cell to cell, so one feature has copies in the neighbouring cells; the **copies** setting applies to all three tools: **all copies** (the default for a blob) hides or keeps the same density in every cell, and **this copy only** (the default for a ball) hides or keeps just the copy you clicked. The two tools remember their own choice, and each rule in the list says which it is. The rules are listed; × brings one back, **Undo** removes the last, and **Reset all** shows the whole map again. The rules belong to the reflection file that is loaded.

- **Count electrons** is a separate subsection with its own red **Count mode** button (only one of the two modes can be on, it turns off when the submenu is closed, and a click in Count mode never hides anything). With it on, clicking a surface counts the electrons in **the blob clicked** or in **a ball around the click**; the button **Count voids** does not need a click. Each adds a row to a table: the region, its volume, the number of electrons, and the raw integral. The **blob** is the connected density above **count to** (a level lower than the one drawn, so that the edges of the density are counted); the **ball** has the **radius** above; **Count voids** takes every grid point farther than the van der Waals radius of each atom plus the **probe** from all atoms (the atoms of the structure with all their symmetry equivalents), splits that free space into separate voids (voids under 20 $Å^3$ are ignored) and counts each, with the total for the cell. The electrons are the integral of the refined guest density over the region. The refinement works as in SQUEEZE and the solvent masks of other programs: one pass of the difference map is only a first-order estimate (it takes the phases of the model without the guest, and it lacks the reflections that were not measured, including F(000), which is the number of electrons), so the guest is refined. Its structure factors are added to the calculated ones, the phase of the sum is used with the observed amplitude to get a better difference, the density that results is set to zero outside the voids, and the guest structure factors are calculated again from it. The reflections that were not measured, and F(000), are given only by the density inside the voids. It settles in 10 to 30 cycles (a second or two). The column **1 pass** is the plain integral of the difference map over the region, for comparison; it is always lower. The voids are the volume the probe can cover, as in other programs: every point within the probe radius of a place where the centre of the probe fits (farther than the van der Waals radius plus the probe from every atom); a bigger **probe** gives smaller voids. Atoms with an occupancy below 0.5 (a modelled guest or solvent, which the .fcf of a guest-omitted refinement does not have either) are left out when the voids are found, and the status line says how many; delete other atoms that sit in the void, or they will fill it. The guest is taken to lie inside the voids, so a blob or ball is counted only where it is in a void; the blob is also kept inside its void, so that at a low level it cannot leak into the framework. **the void clicked** counts the whole void under the pointer. **Clear counts** empties the table.
- **Peaks and holes** lists the highest peaks and deepest holes of the unit cell (symmetry-equivalent ones once), with their height, their height in r.m.s. units, their fractional coordinates and the nearest atom (symmetry included) with its distance, as in the SHELXL listing.
- **Redraw** calculates and draws the surfaces again with the settings above (after Grow, a new selection or a change of region); **Clear** removes the surfaces from the view but keeps the map and the menu, so settings can be changed and the map drawn again. The reflection file is dropped only when another structure is loaded.

HOW THE MAP IS CALCULATED

The unique reflections are expanded by the space-group operators (the phase of the equivalent reflection $h' = R^{T}h$ is the phase of h minus $2\pi\, h{\cdot}t$) and by Friedel's law, put on a reciprocal grid, and transformed with a 3D fast Fourier transform on a grid of about a third of the resolution limit; the amplitude is $|Fo| - |Fc|$ with $|Fo|$ the square root of Fo^2 (zero when Fo^2 is negative), and the result is divided by the cell volume. The map has no F(000) term, so its mean is zero. The surface is drawn from a trilinear interpolation of that grid. For the test structure (MeCN guest left out, $P2_1/c$) the peak of the guest comes at the same place as SHELXL's Q1 and 3.4 Å from the same hydrogen atom; the r.m.s. is 0.142 against SHELXL's 0.138 e/Å³, and the highest value 2.59 against 2.33 (SHELXL reads its peak heights from a coarser grid, and at SHELXL's own grid position the map here is 2.41). Use the map to see where density is and how much, not for the last digit.

WHAT THE ELECTRON COUNTS NEED, AND HOW GOOD THEY ARE

The refinement recovers the reflections that were not measured from the density in the voids, but it can only do that to a point: the lowest-angle reflections carry the broad features of a guest. CrystalVista therefore reports under the table how many of the reflections the space group allows with d above 5 Å are in the file (symmetry equivalents counted once, systematic absences left out), and warns when it is under 90%. Two tests: (1) **20bar_a** (95% of the low-angle reflections present; two voids of 94 Å^3 at probe 1.15): 16.0 electrons in each, which is what the Olex2 solvent mask gives for the same data (96.4 Å^3, 16.0 electrons each, and about 0.3 electrons in each of four small voids, as here); one pass of the difference map gave 11.0. (2) **The MeCN structure** (one acetonitrile molecule, 22 electrons, in each cavity; only 35% of the low-angle reflections present): 18.5 electrons in each of two cavities of 138 Å^3 at probe 1.2, the same within 1.5% for probes of 1.0 to 1.3, against 1.4 to 3.6 from one pass. With so many low-order reflections missing the count is still about 15% low, which the warning says. The numbers also depend on the region: choose the probe to match the question (for example the probe that gives the void volume of Void / Channel Analysis or of another program).

LIMITS

Only `LIST 6` files are read at present. The map is Fo − Fc; a 2Fo − Fc map is not offered. Reflections that SHELXL marks as unobserved or systematically absent in the file are skipped; all others, including weak ones, are used with equal weight, so the noise level is that of an unweighted map. A solvent mask (SQUEEZE or similar) that is already part of Fc in the .fcf is simply included in it.

Appendix A — Keyboard shortcut reference

Key	Action
a	Toggle "Show ASU only" (CIF-based structures)
c	Toggle the unit cell
k	Toggle the axis key
h	Toggle hydrogen display
l	Toggle atom labels
p	New least-squares plane through the selected atoms (3 or more); clears the selection
Arrow keys	Rotate 5° per press (Shift = 90°)
Page Up / Page Down	Roll 5° per press (Shift = 90°)
- / = or +	Zoom out / in one step
Delete / Backspace	Delete selected atoms, or cut selected bond(s)
Ctrl + C	Generate centroid from selection
Esc	Clear the whole selection
Space	Select all / clear selection
Shift + Space	Invert atom selection
Ctrl + Space	Invert bond selection
Ctrl + X	Hide selected atoms
Ctrl + Z	Undo the last undoable action
Ctrl + U	Unhide all atoms
Left drag	Rotate view
Shift + left drag	Roll view
Alt + left drag	Lasso select
Ctrl + Alt + left drag	Lasso select whole fragments
Shift + Ctrl + left drag	Pan (move the model and its rotation centre)
Right drag (vertical)	Zoom

Display > Immobilise view	Switch off left-drag rotation and right-drag zoom

See Section 5 for the full atom-click modifier-key table (plain/Shift/Ctrl/Alt/combinations), and Section 8 for Walk mode's own separate controls.

Appendix B — Macro command reference

A command's argument(s) follow the command word after a single space. Where an argument is an atom reference, it has the form `name  matrix` — the atom's own CIF/SHELX label, two spaces, then the space-group operator that generated that specific copy in Jones–Faithful notation, e.g. `H7A  1+X,1+Y,Z`.

Selection

Command	Recorded on
`SELECT_ALL / DESELECT_ALL`	Spacebar with nothing / anything selected
`TOGGLE_ALL`	Shift+Spacebar
`SELECT_ATOM name matrix / UNSELECT_ATOM name matrix`	Plain click on an unselected / selected atom
`SELECT_FRAG name matrix / DESELECT_FRAG name matrix`	Ctrl-click
`SELECT_ELEMENT # / DESELECT_ELEMENT #`	Shift-click (# = atomic number)
`SELECT_PART # / DESELECT_PART #`	Shift+Ctrl+Alt-click (.res/.ins only)

View and display

Command	Recorded on
`VIEW_VECTOR h k l`	a/b/c buttons, or [hkl] Go
`VIEW_PLANE h k l`	bc/ac/ab buttons, or Perpendicular-to Go
`VIEW x y z w`	Never automatic — "Add current POV" only
`UNIT_CELL_ON / UNIT_CELL_OFF`	Show unit cell checkbox, or `c`
`ASU_ONLY_ON / ASU_ONLY_OFF`	Show ASU only checkbox, or `a`
`HIDE_SELECTION / UNHIDE_ATOMS`	Hide selection / Unhide all
`CENTER_ALL / CENTER_VISIBLE / CENTER_UNITCELL`	Centering dropdown
`CENTER_SELECTED`	Centering > Selected atoms + Apply
`CENTER x,y,z`	Centering > Fractional coordinates + Apply

Editing the structure

Command	Recorded on
`SELECTED_TO_ASU`	"Move to ASU"

DELETE_SELECTED_ATOMS	"Delete selection"
CREATE_CENTROID	"Generate centroid" / Ctrl-C
NORMALISE_H	"Normalise H atoms" (Structure > Edit bonds): the selected H atoms, or every H atom if none is selected, moved to the X–H distances currently set there
DELETE_SYM_ATOMS	"Delete all symmetry-generated atoms"
REASSIGN_ELEMENT #	"Reassign element to"
MOVE_SELECTION_TO Host / NewGuestGroup / Guest1…	"Move selection to"

Growing by symmetry

Command	Recorded on
GROW_ATOM / GROW_FRAG / GROW_ASU name matrix	Grow > Mouse click
GROW_SPHERE_ATOM / _FRAG / _ASU # name matrix	Grow > Mouse (Sphere of inclusion)
PACK_FRAGATOMS / PACK_FRAGCENTROID amin amax bmin bmax cmin cmax	Pack tab's Pack button
GROW_SYM matrix	Symmetry tab Apply / shift buttons
EXTEND_FRAGS / COMPLETE_MOLS	"Extend fragment(s)" / "Complete molecule(s)"

Atom display style

Recorded together as one batch reflecting Style > Atoms' complete state, only when Apply is pressed.

Command	Meaning
ATOM_STYLE_CPK_COLOR / ATOM_STYLE_CUSTOM_COLOR #rrggbb	Colour choice
ATOM_STYLE_ELLIPSOID / ATOM_STYLE_SPHERE	Ellipsoid checkbox state
ATOM_STYLE_RADIUS_FIXED # / ATOM_STYLE_RADIUS_VDW # #	Radius option (Non-H, H)
ATOM_STYLE_CLEAR_SELECTION	Only when the Clear selection checkbox is ticked
ATOM_STYLE_APPLY	Always last — applies the lines above it

Void/channel analysis

Command	Meaning

`CALCULATE_VOID`	"Calculate", forcing Accurate volumes **ON**
`CALCULATE_VOID_Q`	"Calculate", forcing Accurate volumes **OFF**

Export

Command	Meaning
`EXPORT_PNG suffix`	Save the current view as a PNG (suffix optional). Never automatic — "Add EXPORT_PNG" only.
`SAVE_JSON suffix`	Save the model as a project file (suffix optional). Never automatic — "Add SAVE_JSON" only.

Appendix C — How void/channel space is calculated

This appendix describes the algorithm behind Section 9 in more depth than the in-app Guide does, for anyone who wants to know exactly what a reported region, classification, or volume actually means.

1. Building the void network: a weighted Delaunay tetrahedralization

Given the host atoms (Cartesian position and van der Waals radius each) and, in "Fill unit cell" mode, the unit cell, the app builds **one** periodic void network via a weighted Delaunay (*regular*) tetrahedralization, computed as the lower convex hull of the atoms lifted into 4D (the standard "paraboloid lifting" construction for a weighted Delaunay triangulation). Voids and channels are simply two classifications of that *one* network's connected components — there is no separate code path for either, and critically, **no grid or resolution parameter enters into the topology or the void/channel classification decision at all**. A probe radius only prunes an already-exact graph after the fact; it is never consulted while building it.

> **WHY A REGULAR TRIANGULATION, NOT A PLAIN VORONOI DIAGRAM**
>
> Atoms have unequal radii, so the physically correct "largest empty sphere" network is an additively-weighted (Apollonius) Voronoi diagram, whose cell boundaries are hyperboloids — a substantially harder computational-geometry problem to implement robustly. Following the same approximation used by established tools in this space (e.g. Zeo++), CrystalVista instead uses the *power* (radical) diagram, whose bisectors are ordinary planes, as a computationally tractable proxy: the network's *topology* (which cavities exist and how they connect) comes from the power diagram, but every reported *radius* is the true Euclidean distance to the nearest atom's own van der Waals surface — the physically meaningful quantity — not the power distance itself. The two coincide exactly only when the atoms defining a given vertex happen to share the same radius.

Each tetrahedron of the weighted triangulation corresponds to one vertex of the dual void network (a point equidistant, in the power-distance sense, from its four defining atoms) with a well-defined clearance radius. Two network vertices are joined by an edge when their tetrahedra share a face (three atoms in common); the connected components of this vertex–edge graph, after removing every vertex whose own clearance radius is smaller than the current probe radius, are exactly the candidate voids and channels. A component that closes up on itself within one unit cell is a **void**; a component whose own vertices span a full periodic repeat along one or more cell axes is a **channel**, with its dimensionality equal to the number of axes it is periodic along.

2. Probe-radius stability limits

The **Limit** column (Section 9) is found by binary search directly on this exact graph, not by re-running the whole calculation at many probe radii. For a channel, the search asks "at what probe radius does this component stop being connected across a full periodic repeat" — the largest radius at which it is still classified as a channel. For a void, the search asks "at what probe radius does it first connect to a periodic image of itself" — the smallest radius at which it would channelize. Both searches bisect

between a practical floor (0.05 Å) and the probe radius currently in use, re-testing the component's own connectivity at the midpoint each iteration, converging to the stability limit in a fixed, small number of iterations regardless of network size.

3. From network to a reported volume: grid + Euclidean distance transform

The void network above gives topology and connectivity for free, at no resolution cost, but it does not directly give a swept *volume* — that still has to be measured geometrically. For each component, the app builds a local 3D grid over its own bounding region (padded for periodic completion where relevant), computes each grid cell's Euclidean distance to the nearest atom surface (a distance transform, not a naive per-cell atom scan), and counts every cell whose distance meets or exceeds the current probe radius as "swept". The reported volume is that cell count times the cell volume; the same distance field also drives the isosurface actually drawn in the 3D view (at distance = probe radius), so the number shown and the picture drawn come from the same field and normally agree closely; they are not identical, since an extrapolated volume is a fit while the drawn surface is a finite-resolution, smoothed mesh.

Because this is a discretization, a coarser grid resolution changes the count — a voxel straddling the true boundary is either counted whole or not at all, which can err in either direction, and the effect is proportionally larger for a small cavity than a large one (more of its volume sits near the boundary relative to its interior). Extrapolating to zero spacing is meant to remove this.

4. Accurate volumes: Richardson extrapolation to zero grid spacing

Accurate volumes (extrapolated, slower) addresses this directly rather than simply using a finer fixed grid. Each component is measured at six progressively finer grid spacings, and a straight line is fit to volume against spacing ($V = V_\infty + a \cdot h$); the reported value is the fitted V_∞ — the volume the grid method would converge to at an (unreachable) zero spacing — with an uncertainty derived from the fit's own residual and its distance from the two finest directly-measured points. This is the same underlying quantity a rolling-probe program such as MSRoll reports directly, without needing a grid at all, and was compared with MSRoll's output during development on one real structure, where after linear extrapolation the two methods agreed to within roughly 1–1.5%, a gap that may reflect the two methods defining "swept volume" slightly differently. That is a single comparison, not a systematic benchmark, and the fit's uncertainty does not include the choice of atomic radii or the network approximation. A quadratic extrapolation model was tried and rejected during development — it moved the fitted value slightly *further* from the reference while making the reported uncertainty look smaller, which would have been actively misleading rather than more accurate; the straight-line model is used because it is the one whose own uncertainty estimate can actually be trusted.

5. What "Fill unit cell" completes, and what periodicity means for a channel's own picture

In "Fill unit cell" mode the atom set fed into steps 1–4 above is the host completed by the CIF's own symmetry across a padded neighbourhood of unit cells, not just whatever happens to be grown on screen — so the calculation is always run against the true, complete periodic structure regardless of how much of it is currently displayed. A channel's *rendered* surface, by contrast, is deliberately just one

period's worth of wall by default (extra periodic copies are only added where display or Walk-mode collision genuinely need them, and even then only as a rendering/collision convenience, never by re-deriving the underlying network) — use the Fragments panel's Move/Copy to add further real, solid periodic copies for a figure or for Walk-mode exploration further along a channel's own length.

> **IN SHORT**
>
> Classification (void vs. channel, and the Limit column) is worked out exactly, and independently of any grid, on the geometric network described in step 1. For atoms of different radii that network is a power-diagram approximation of the true rolling-probe accessibility, so the classification is exact for the network but not a proof of physical accessibility. Only the reported *volume* is a numerical measurement with a controllable error, and Accurate volumes exists specifically to quantify and remove that error rather than just guessing at a finer grid.